SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Code of Conduct:

I _P. Lakshmi Gayathri Navaneetha_ (192321169) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

P. Gayathri
Signature of the student:

ANSWERS

Question 1: ATM Withdrawal Module

Scenario: Only input-output behavior of a partially tested ATM withdrawal module is known (limits, balance validation, error messages).

Observed behavior (assumed from the black-box observations): withdrawals of ₹500, ₹5,000 and ₹20,000 succeed; ₹150 shows 'Enter amount in multiples of 100'; ₹25,000 shows 'Exceeds per-transaction limit'; an amount above the balance shows 'Insufficient funds'; a third withdrawal that passes ₹50,000 in one day shows 'Daily limit exceeded'. Assumed rules inferred from this: minimum ₹100, multiples of ₹100, per-transaction maximum ₹20,000, daily limit ₹50,000.

1. Decomposition into functional components

Component	Function	Observed output
C1 Amount format check	Numeric, positive, multiple of 100	'Invalid amount' / 'Enter amount in multiples of 100'
C2 Limit check	Minimum 100, per-transaction maximum 20,000	'Exceeds per-transaction limit'
C3 Balance check	Amount ≤ available balance	'Insufficient funds'
C4 Daily limit check	Today's total + amount ≤ 50,000	'Daily limit exceeded'
C5 Cash availability	ATM has enough notes	'Cash not available'
C6 Dispense and ledger update	Dispense, debit balance, print receipt	Success message

2. Equivalence classes and boundary conditions identified

Input	Valid classes	Invalid classes	Boundary values
Amount	100 to 20,000 in multiples of 100	< 100; > 20,000; not a multiple of 100; zero/negative; non-numeric	0, 99, 100, 101, 19,900, 20,000, 20,100
Balance vs amount	Amount ≤ balance	Amount > balance	balance = amount; balance = amount − 1
Daily total	Total ≤ 50,000	Total > 50,000	49,900 + 100; 50,000 + 100
ATM cash	Cash ≥ amount	Cash < amount	cash = amount; cash = amount − 100

3. Inferred techniques and adequacy

- **ECP (partial):** one value per class was observed (500, 5,000, 25,000, 150), so the main classes were identified.
- **BVA (weak or absent):** only 20,000 appears to have been tried. Values such as 99, 100, 20,100 and balance = amount were not observed.
- **Error guessing:** non-multiple of 100 suggests an intuition-based test.
- **Gaps:** no multi-condition cases (for example balance sufficient but daily limit exceeded), no order-of-validation checks, no cash-availability cases, no zero or negative amounts.

4. Reconstructed improved test cases

TC	Amount / condition	Class / boundary	Expected result
1	99	Below minimum	Rejected (invalid)
2	100	Minimum	Dispensed

TC	Amount / condition	Class / boundary	Expected result
3	101	Just above minimum (not a multiple)	Rejected: multiples of 100
4	20,000	Per-transaction maximum	Dispensed
5	20,100	Above maximum	Rejected: exceeds limit
6	0, −100	Invalid amount	Rejected
7	abc, blank, 100.50	Invalid format	Rejected, no crash
8	Balance 5,000; withdraw 5,000	Balance = amount	Dispensed, balance 0
9	Balance 5,000; withdraw 5,100	Balance < amount by 100	Rejected: insufficient funds
10	Today's total 49,900; withdraw 100	Daily total = 50,000	Dispensed
11	Today's total 49,900; withdraw 200	Daily total > 50,000	Rejected: daily limit
12	Amount 20,000; balance 10,000	Two failures together	Defined message priority (consistent with requirement)
13	ATM cash 3,000; withdraw 3,000 / 3,100	Cash boundary	Dispensed / 'Cash not available'
14	Withdraw and cancel; network failure after debit	Transaction integrity	No debit without dispense (or auto-reversal)

5. Justification

The reconstructed set covers each component, each equivalence class, and each boundary on both sides, and adds multi-condition and failure-handling cases that the observed behavior did not show. 14 test groups give complete coverage of the inferred logic, compared with the handful of cases observed.

Question 2: Execution Traces and Code Snippets (No Documentation)

Scenario: Execution traces and code are available, test documentation is not. Reverse engineer the white-box approach and reconstruct a full path testing strategy.

1. Code snippet reconstructed from the traces (assumed)

Line	Code
1-3	read n, items[]; total = 0; i = 0
4	while (i < n) [D1]
5	if (items[i].qty <= 0) { i++; continue; } [D2]
6	price = items[i].unitPrice × items[i].qty
7	if (price > 1000 && member) price = price × 0.9 [D3, two conditions]
8-9	total += price; i++
10	if (total > 5000) total = total − 200 [D4]
11	return total

2. Control Flow Graph

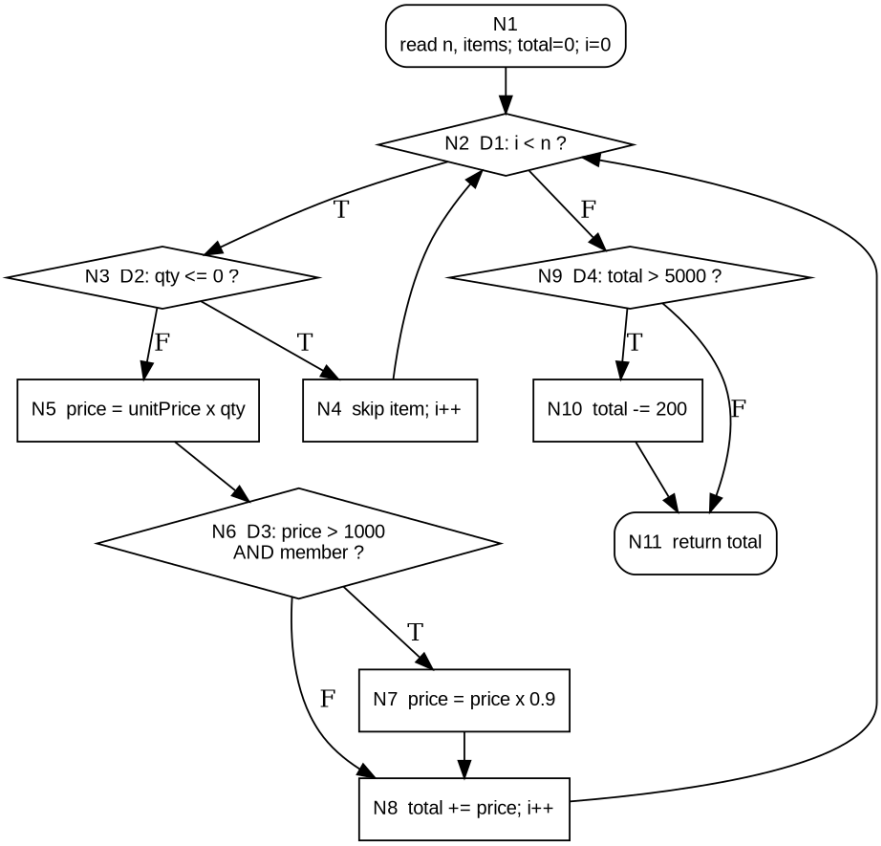


Figure 1: CFG of the module (D1 to D4 are decision nodes)

Decision points: D1 (loop), D2, D3 (compound: price > 1000 AND member), D4. **Loop:** one (N2 to N8 back to N2).

Cyclomatic complexity: V(G) = E − N + 2 = 14 − 11 + 2 = **5** (also 4 decision nodes + 1).

3. Independent paths

Path	Nodes	Meaning	Test data
P1	N1-N2-N9-N11	Loop not entered	n = 0, expect total 0
P2	N1-N2-N3-N4-N2-N9-N11	Item skipped (qty ≤ 0)	n = 1, qty = 0, expect total 0
P3	N1-N2-N3-N5-N6-N8-N2-N9-N11	Normal item, no discount	price 500 × 1, non-member, expect 500
P4	N1-N2-N3-N5-N6-N7-N8-N2-N9-N11	Member discount	price 1,500 × 1, member, expect 1,350
P5	N1-N2-N3-N5-N6-N8-N2-N9-N10-N11	Total above 5,000	600 × 10, non-member, expect 5,800

4. Evaluation of existing test adequacy (from traces)

Given traces: TR1 (n = 2, prices 500 and 600, no member), TR2 (n = 1, price 1,500, member), TR3 (n = 0).

Criterion	Result	Gap
Statement coverage	9 of 11 nodes ≈ 82%	N4 (skip) and N10 (−200) never executed
Branch coverage	6 of 8 branches = 75%	D2 true and D4 true never taken
Path (basis) coverage	3 of 5 = 60%	P2 and P5 missing
Condition coverage (D3)	member = false with price > 1000 never tested	Masked-condition faults possible

Inference: the original tests look like a functional happy-path set with incidental coverage. No sign of structural criteria such as branch or path coverage.

5. Reconstructed path testing strategy

- Run the 5 basis-path tests above (P1 to P5): gives 100% statement, 100% branch and 100% basis-path coverage.
- Loop testing on D1: n = 0, 1, 2, typical, maximum, maximum + 1.
- MC/DC on D3: (price > 1000, member) = (T,T) gives T; (F,T) gives F with price 1,000, member; (T,F) gives F with price 1,001, non-member.
- Boundary tests: price 1,000 / 1,001; total 5,000 / 5,001; qty −1, 0, 1.
- Interaction path: a skipped item followed by a discounted item, then total > 5,000.

Reasoning: the five paths form a basis, so any other path is a combination of them, and loop plus MC/DC tests cover the faults that path coverage alone cannot see.

Question 3: Web Login and Transaction System (Defect Logs Only)

Scenario: Only defect logs and test results are available. Reverse engineer the testing approach and reconstruct a comprehensive strategy.

1. Defect pattern analysis (assumed log extract)

Defect	Pattern	Technique it suggests was used	Technique that was missing
D1	Password of 7 characters accepted; minimum is 8	BVA on password length (found)	Boundary 8 and 9 not paired
D2	Account locked after 4 attempts instead of 5	Boundary on attempt counter (found)	Counter reset path not tested
D3	Transfer of exactly ₹10,000 rejected (limit is ₹10,000)	Boundary on limit (found late)	Limit −1 / +1 not systematically tested
D4	Negative transaction amount accepted	Error guessing (found)	ECP invalid classes
D5	Duplicate transaction on double click	Exploratory testing	State and idempotency tests
D6	Session still valid after logout	Exploratory / error guessing	State-based and path tests
D7	Failure counter not reset after successful login	Code-level path defect	Path testing on login CFG
D8	Special characters in remarks cause server error	Error guessing	Negative / input-validation classes

Inferred techniques used: a mix of ad-hoc BVA, error guessing and exploratory testing. Defects D5 to D7 are path and state defects, which indicates that no structural (white-box) path testing was done.

2. Decomposition into testable components

Component	Test focus
Authentication	Password rules, attempt counter, lockout, reset
Session management	Timeout, logout, token invalidation
Transaction input validation	Amount, account number, remarks
Transaction processing	Balance check, limits, idempotency, rollback
Audit and logging	Entry for every success and failure

3. Gaps in coverage

- No systematic equivalence classes for invalid input (negative, zero, non-numeric, special characters).
- BVA done only on some fields; limits tested after failure, not by design.
- No path testing of the login logic (lockout and reset paths).
- No state-based tests (session, double submission).

- No traceability from requirement to test, so gaps are invisible.

4. Reconstructed comprehensive strategy

Technique	Application	Sample tests
ECP	Username/password, amount, account number	Valid, empty, too short, too long, special characters, negative, zero, non-numeric
BVA	Password length 7/8/9; attempts 4/5/6; amount 0/1/9,999/10,000/10,001	One test on each side of every boundary
Path testing	Login CFG: decisions are user found, password valid, counter ≥ 5, account locked; V(G) = 5	5 basis paths incl. failure then success (counter reset)
Decision table	Login result vs credentials valid, account locked, session active	One test per rule
State testing	Session: logged out, active, expired	Logout then reuse token; double click submit
Security / error guessing	Special characters, repeated attempts	Rejected safely, no server error

5. Justification

Each defect class in the log maps to a technique in the reconstructed strategy: boundary defects to BVA, invalid input to ECP, counter/reset to path testing, and session and duplicate defects to state tests. Traceability from defect to test makes the coverage verifiable and prevents recurrence.

Question 4: Legacy Inventory Management: Stock Update Logs

Scenario: Only successful and failed transaction logs for stock updates and low-stock alerts are available.

1. Log extract (assumed) and inferred rule

Log	Stock before	Operation	Result	Alert
L1	100	Sale 30	Success, stock 70	None
L2	70	Sale 49	Success, stock 21	None
L3	70	Sale 50	Success, stock 20	LOW_STOCK
L4	20	Sale 20	Success, stock 0	OUT_OF_STOCK
L5	0	Sale 1	Failed: insufficient stock	-
L6	50	Sale 0 / Sale −5	Failed: invalid quantity	-
L7	20	Receive 5	Success, stock 25	Alert cleared
L8	9,990	Receive 20	Failed: exceeds capacity 10,000	-

Inferred hidden rules: quantity must be ≥ 1; a sale cannot exceed stock; LOW_STOCK alert triggers when remaining stock ≤ 20 (21 gives no alert, 20 does); OUT_OF_STOCK at 0; stock capacity is 10,000; an alert clears when stock rises above 20.

2. Functional decomposition

Unit	Function
U1 Quantity validation	Integer ≥ 1
U2 Availability check	Sale quantity ≤ current stock
U3 Stock update	Add (receive) or subtract (sale)
U4 Capacity check	New stock ≤ 10,000
U5 Alert evaluation	Set or clear LOW_STOCK / OUT_OF_STOCK
U6 Transaction logging	Record success or failure with reason

3. Equivalence classes and boundaries inferred

Parameter	Classes	Boundaries
Quantity	< 1 (invalid); ≥ 1 (valid)	0, 1
Sale vs stock	qty ≤ stock; qty > stock	stock − 1, stock, stock + 1
Remaining stock	0; 1 to 20 (low); ≥ 21 (normal)	0, 1, 20, 21
Receive vs capacity	new stock ≤ 10,000; > 10,000	9,999, 10,000, 10,001

4. Reconstructed black-box test design (covering gaps in the logs)

TC	Stock	Operation	Expected	Why (inferred logic)
1	100	Sale 1	99, no alert	Minimum valid quantity
2	100	Sale 79	21, no alert	Alert boundary: just above 20
3	100	Sale 80	20, LOW_STOCK	Alert boundary: exactly 20
4	100	Sale 81	19, LOW_STOCK	Just below boundary
5	100	Sale 99 / 100	1 LOW_STOCK / 0 OUT_OF_STOCK	Zero boundary
6	100	Sale 101	Failed: insufficient stock	stock + 1
7	50	Sale 0 / −1	Failed: invalid quantity	Quantity boundary
8	20	Receive 1	21, alert cleared	Alert clearing boundary (not in logs)
9	9,999	Receive 1 / 2	10,000 success / failed capacity	Capacity boundary
10	0	Receive 100	100, OUT_OF_STOCK cleared	Recovery from zero
11	20	Two simultaneous sales of 15	One succeeds, one fails; stock never negative	Concurrency gap

5. Justification

Every test is derived from a rule inferred from the logs. The logs show alert behavior at only a few points, so tests are added on both sides of each hidden boundary (20/21, 0, 10,000) and for situations the logs never recorded (alert clearing, recovery from zero, concurrency). Together they show whether the inferred logic is correct and whether defects hide at the edges.

Question 5: Library Management .exe and CPU/Memory Profile

Scenario: A compiled library management module and its stress-test CPU/memory profile are available. Map the spikes to loops and decision clusters and construct a path testing strategy.

1. Mapping execution spikes to likely logic (assumed profile)

Observation in profile	Likely logic	Inference
Memory rises linearly during start-up with catalog size	Loop L1 loading books into an index	O(n) loop; memory proportional to catalog
CPU stays near 100% and duration grows roughly with members × loans	Nested loops L2 (members) and L3 (loans)	O(n × m) behavior; potential bottleneck
Short periodic CPU bursts during overdue processing	Decision cluster D1 (overdue?) and D2 (> 30 days?) with fine calculation	Branch-heavy region
Memory sawtooth that does not return to baseline after report generation	Loop L4 writing report rows and allocating buffers	Possible memory leak or unreleased buffers

2. Conceptual Control Flow Graph

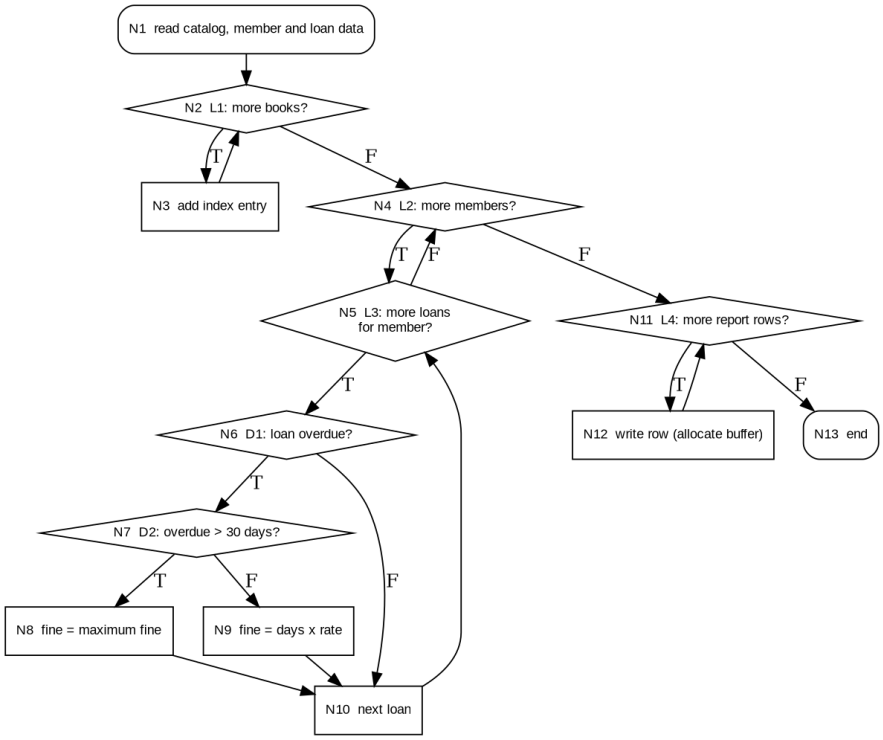


Figure 2: Conceptual CFG reconstructed from the profile (L = loop, D = decision)

Predicate nodes: L1, L2, L3, D1, D2, L4 = 6. **V(G) = 6 + 1 = 7** (check: E − N + 2 = 18 − 13 + 2 = 7).

3. Path testing strategy targeting resource-intensive paths

Path	Condition outcomes	Test load	Measure
B1	L1 F, L2 F, L4 F	Empty data (baseline)	Idle CPU and memory
B2	L1 T (loop entered)	Catalog of 1, 10^3, 10^6 books	Memory growth per book (linear?)
B3	L2 T, L3 F	Members with no loans	CPU per member
B4	L3 T, D1 F	Many loans, none overdue	Cheap-path cost
B5	D1 T, D2 F	Overdue ≤ 30 days	Fine path 1 cost
B6	D1 T, D2 T	Overdue > 30 days	Maximum-fine path cost

Path	Condition outcomes	Test load	Measure
B7	L4 T	Report of 10^5 rows, repeated 20 times	Memory returns to baseline?

- Add loop-boundary tests for each loop (0, 1, 2, typical, maximum, maximum + 1).
- Add combined stress path: 10,000 members × 50 loans each, all overdue, report generated, repeated for a soak period (for example 2 hours).
- Use a profiler and sampling to confirm that CPU time follows the predicted nested-loop growth, and that memory returns to baseline after each run.

4. Impact on system stability

- Detects quadratic growth early, so data-size limits and indexing/batching changes can be designed before production.
- Soak repetition of B7 exposes leaks that would eventually cause out-of-memory crashes.
- Targeting each path separately shows which branch cluster contributes to the CPU peaks.
- Result: defined performance thresholds per path and a regression suite to protect them.

Question 6: Mobile Payment Gateway: State Machine

Scenario: API JSON responses for Pending, Success, Reversal and Timeout are available.

1. Reverse-engineered state machine

States: PENDING, SUCCESS, TIMEOUT, REVERSAL (terminal). **Triggers inferred from JSON:** a response with status PENDING after creation; status SUCCESS with bank reference; status TIMEOUT when the bank does not respond within the time limit; status REVERSAL with reversalReason when funds are returned.

Data constraints inferred: reversal amount ≤ original amount; the same transaction ID returns the same final state (idempotent); a reversal requires the original to be SUCCESS or TIMEOUT; reversalReason is mandatory.

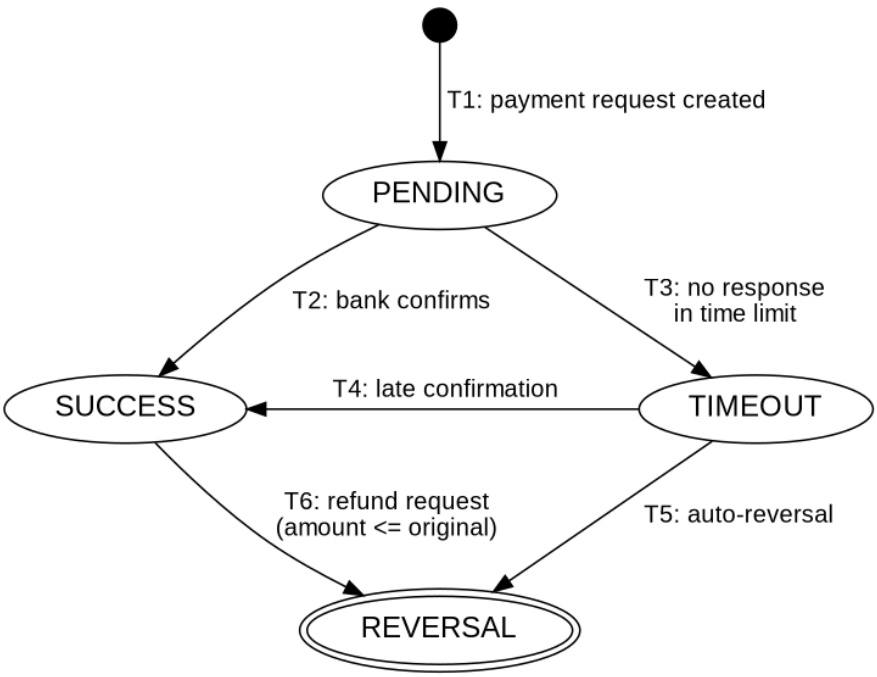


Figure 3: State-transition diagram of the payment gateway

2. Technique inferred

The responses are consistent with **State Transition Testing**: each status is a state and each trigger an event. The implicit approach was probably 0-switch (each valid transition once); the illegal transitions were not tested.

3. Legal transitions (0-switch coverage)

ID	From	Event	To	Expected JSON
T1	Start	Create payment	PENDING	status: PENDING, txnId present
T2	PENDING	Bank confirms	SUCCESS	status: SUCCESS, bankRef
T3	PENDING	No response in time limit	TIMEOUT	status: TIMEOUT
T4	TIMEOUT	Late confirmation	SUCCESS	status: SUCCESS
T5	TIMEOUT	Auto-reversal	REVERSAL	status: REVERSAL, reason: TIMEOUT
T6	SUCCESS	Refund request	REVERSAL	status: REVERSAL, amount ≤ original

4. Illegal transitions (must be rejected, state unchanged)

ID	From	Event	Expected result
I1	SUCCESS	Back to PENDING	Error: invalid transition
I2	SUCCESS	Timeout event	Ignored; state stays SUCCESS
I3	TIMEOUT	Back to PENDING	Error: invalid transition
I4	REVERSAL	Success confirmation	Error: transaction closed
I5	REVERSAL	Second reversal	Error: already reversed
I6	REVERSAL	Timeout / pending event	Error or ignored
I7	Start	Direct SUCCESS (skip PENDING)	Error: invalid transition
I8	Start	Direct REVERSAL	Error: no original payment

5. Sequences and data constraint tests

- **1-switch sequences:** T1-T2-T6 (pay then refund); T1-T3-T4 (timeout then late success); T1-T3-T5 (timeout then auto-reversal); T1-T2 then I1.
- **Data constraints:** reversal amount = original (accept), original + 0.01 (reject), partial reversal; missing reversalReason (reject).
- **Idempotency:** repeat the same T2 request twice; same txnId must not charge twice.
- **Race:** late confirmation (T4) arriving at the same time as auto-reversal (T5); only one may win and the amount must be consistent.

All 6 legal and 8 illegal transitions are covered, so the suite has 100% transition coverage plus negative coverage.

Question 7: Encrypted Communication Module: Fuzzing and BVA

Scenario: Only input parameters and encrypted outputs are observable. Reverse engineer validation by fuzzing and reconstruct BVA test cases.

1. Decomposition

Component	Function	Test focus
Data validation	Checks message length/characters, key format, recipient ID	Error-handling boundaries
Encoding / normalization	Converts input to bytes (e.g., UTF-8)	Multi-byte characters, null bytes
Encryption engine	Encrypts with key and mode	Correct ciphertext, padding, key length
Output formatting	Returns ciphertext and status	No leakage of internal errors

2. Fuzzing analysis: inferred validation boundaries (assumed observations)

Field	Fuzz result	Inferred rule
Message	1,024 characters accepted; 1,025 rejected 'Too long'; empty rejected	Length 1 to 1,024
Key	32 hex characters accepted; 31 and 33 rejected 'Invalid key'; non-hex rejected	Exactly 32 hex characters
Recipient ID	20 alphanumeric characters accepted; 21 rejected; special characters rejected	1 to 20, alphanumeric
Special input (quotes, markup, null byte)	Accepted inside message; rejected in ID and key	Whitelist on ID/key; message treated as data

3. Reconstructed BVA test cases

Field	Test values	Expected
Message length	0, 1, 2, 1,023, 1,024, 1,025	Reject, accept, accept, accept, accept, reject
Key length	0, 31, 32, 33	Reject, reject, accept, reject
Recipient ID length	0, 1, 20, 21	Reject, accept, accept, reject
Robustness extremes	65,535, 65,536, $2^{31} - 1$ characters	Rejected safely, no crash, bounded memory
Key characters	32 characters with one non-hex (g, space, null)	Reject

4. Injection and overflow-oriented tests

Category	Input examples	Expected secure behavior
Buffer overflow	Over-length input at each field; very long single token; repeated max-size requests	Length checked before copy; clean error; no crash or truncation
SQL / command injection	Quote characters, SQL keywords, shell metacharacters in the ID and message	Treated as data; parameterized queries; no execution
Script injection	HTML/script tags in message and ID	Encoded or rejected; never executed
Format string	Sequences such as %s %n in message	Printed literally; no memory access
Encoding tricks	Null byte, overlong UTF-8, mixed Unicode	Normalised before validation; rejected if invalid

5. Justification

Boundary tests on every length limit show that the validation check happens before any memory copy, which is how buffer overflows are prevented. Whitelisting (ID and key) and treating message content strictly as data prevent injection. Fuzzing continues as a regression activity because it generates unexpected inputs that a designed suite will not contain.

Question 8: Insurance Premium Calculator: 100% Statement Coverage but Defects

Scenario: Legacy unit tests show 100% statement coverage but miss multi-condition if defects.

1. Reverse engineering the original approach

Code (assumed): premium = base; D1: if (age < 25 AND accidents > 0) × 1.5; D2: if (smoker OR bmi > 30) × 1.3; D3: if (noClaimYears ≥ 5 AND NOT highRisk) × 0.9.

One test with every decision true (age 22, 1 accident, smoker, 6 no-claim years, not high risk) executes every statement, so statement coverage is **100%**. That suggests the team used **statement coverage as the only criterion**. It is inadequate: all false branches (3 of 6), all individual conditions, and 7 of 8 paths are untested.

2. Control Flow Graph

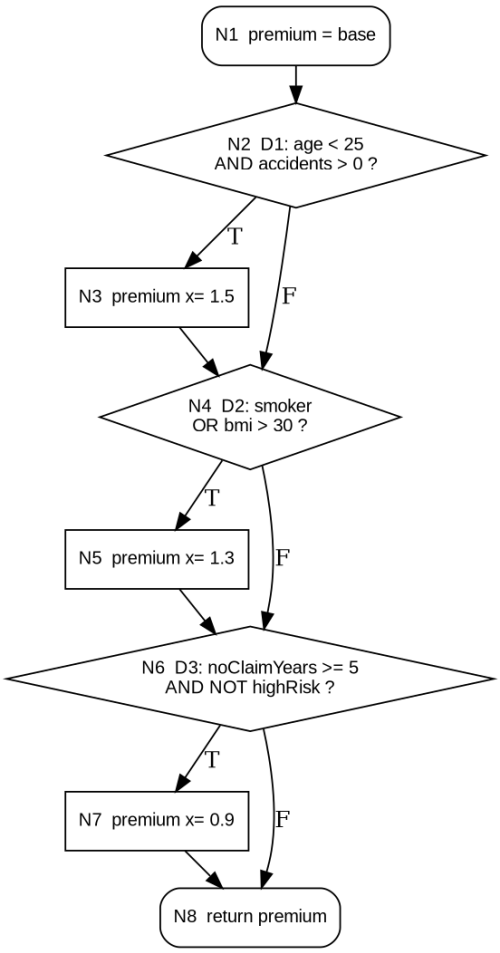


Figure 4: CFG of the premium calculator

Decision nodes D1, D2, D3 → **V(G) = 4** (E − N + 2 = 10 − 8 + 2). Three sequential independent decisions give 2^3 = **8 distinct paths**. Basis set (4 paths): FFF, TFF, FTF, FFT.

3. Path testing strategy (100% branch and 100% path coverage), base premium = 10,000

Path	D1	D2	D3	Inputs	Expected premium
1	T	T	T	age 22, acc 1, smoker, bmi 22, noClaim 6, highRisk N	17,550
2	T	T	F	age 22, acc 1, non-smoker, bmi 32, noClaim 2, highRisk N	19,500

Path	D1	D2	D3	Inputs	Expected premium
3	T	F	T	age 22, acc 1, non-smoker, bmi 22, noClaim 6, highRisk N	13,500
4	T	F	F	age 22, acc 1, non-smoker, bmi 22, noClaim 6, highRisk Y	15,000
5	F	T	T	age 30, acc 1, smoker, bmi 22, noClaim 6, highRisk N	11,700
6	F	T	F	age 22, acc 0, non-smoker, bmi 32, noClaim 2, highRisk N	13,000
7	F	F	T	age 30, acc 1, non-smoker, bmi 22, noClaim 6, highRisk N	9,000
8	F	F	F	age 22, acc 0, non-smoker, bmi 22, noClaim 2, highRisk N	10,000

The eight tests achieve 100% statement, 100% branch and 100% path coverage. The input values are also chosen so that each individual condition is shown to change its decision (for example D1 false via age ≥ 25 in test 5 and via accidents = 0 in test 6; D2 true via smoker in test 1 and via bmi in test 2; D3 false via noClaim < 5 in test 2 and via highRisk in test 4), which gives MC/DC as well. If business rules make some combination impossible (for example accident and 6 claim-free years together), document it as infeasible and replace it with the nearest valid case.

4. Boundary values for the conditions

- age: 24, 25; accidents: 0, 1; bmi: 30, 31; noClaimYears: 4, 5.

5. How this reduces calculation risk

- Every combination of the multipliers (1.5, 1.3, 0.9) is calculated and checked against an expected value, so wrong multiplication order or a missed multiplier is caught.
- Individual condition tests catch wrong operators (AND coded as OR, > coded as ≥).
- Boundary tests catch off-by-one errors in age, BMI and years.
- Because premium errors are financial and regulatory risks, full path coverage is a justified exit criterion.

Question 9: Cloud File Synchronization: Lost Files

Scenario: User activity logs and 'lost file' defect reports are available. Infer the sync logic and reconstruct a revised strategy.

1. Defect pattern analysis (assumed)

Pattern in defect reports	Inferred cause in the algorithm
Files > 2 GB lost on slow networks (< 1 Mbps)	Upload timeout (e.g., 30 s per chunk or per file) treated as success; local copy removed or marked synced
Files of 0 bytes never appear	Empty file skipped by size check (size > 0 rather than ≥ 0)
Temporary or locked files (.tmp, open databases) lost	File type exclusion list or lock handling drops the file silently
File exactly at chunk size (e.g., 4 MB) loses its last chunk	Off-by-one in chunk count calculation
Loss after network drop mid-transfer	Retry limit (e.g., 3) exceeded, then file marked complete or abandoned without alert
Two edits with the same timestamp, one overwritten	Conflict rule uses timestamp only

2. Inferred ECP used by developers

Parameter	Classes likely used	Missing classes / boundaries
File size	Small (< 1 MB), medium (1 MB to 1 GB), large (> 1 GB)	0 bytes; exact chunk size ± 1; 2 GB ± 1; very large (> 10 GB)
File type	Documents, images, videos	Temporary, locked, hidden, symbolic links, long file names
Network speed	Fast (> 10 Mbps), normal	Slow (< 1 Mbps), intermittent, offline during transfer

3. Revised strategy: BVA integrated with stress testing

Area	Test values / scenarios	Expected
Size BVA	0 B, 1 B, 4 MB − 1, 4 MB, 4 MB + 1, 2 GB − 1, 2 GB, 2 GB + 1	All files fully synced, checksum identical
Network speed BVA	Throttle to 0.5, 1, 1.1, 10 Mbps	Slow transfers finish or are retried; none marked complete when incomplete
Failure injection	Drop connection at 0%, 50%, 99% and just before the final acknowledgement	Resume or restart; local file kept until server confirms
Retry boundary	Failures 2, 3, 4 times consecutively	After the limit the file is flagged and the user is notified, never silently lost
Stress: volume	100,000 small files; 1,000 large files in parallel	No missing files; CPU/memory stable
Stress: duration	48-hour continuous sync with random edits and disconnects	File counts and checksums match at the end
Stress: resources	Disk almost full; low memory; battery saver	Clear error, no deletion
Type and concurrency	Locked, temporary, long-named files; two clients editing the same file with the same timestamp	Defined behavior (retry later / conflict copy), no loss

4. Justification: preventing future data loss

- BVA covers the exact boundaries where the lost-file defects were found.
- Stress and failure-injection tests recreate real network and load conditions that normal functional tests do not.
- The central verification is an invariant: source and destination checksum match, and no local file is removed before server acknowledgement. This invariant is checked in every test.
- Automated nightly runs and a defect-to-test rule (each lost-file defect becomes a permanent test) stop regressions.

Question 10: E-Commerce Search and Filter: SQL Queries

Scenario: SQL queries generated by the module are available, but no source code.

1. Query analysis (assumed sample)

Query	WHERE clause observed	Inference
Q1	category = 'Shoes' AND price BETWEEN 1000 AND 5000	Category and price range combined with AND; BETWEEN is inclusive
Q2	category = 'Shoes' AND brand IN ('Nike','Adidas') AND in_stock = 1	Multiple brands combined with OR inside the group; groups joined by AND
Q3	(brand = 'Nike' OR brand = 'Puma') AND rating >= 4	Parentheses used for the OR group
Q4	category = 'Shoes' AND NOT (color = 'Black')	Colour exclusion implemented with NOT
Q5	price <= 2000 AND discount > 0 OR free_shipping = 1	Missing parentheses: AND binds before OR, so free-shipping items bypass the price filter (possible defect)

Decision-making logic inferred: filters of different types are combined with AND; values within one filter are combined with OR (IN); exclusions use NOT; an unselected filter adds no condition (it is omitted).

2. Conditions for the decision table

Condition	Meaning	SQL fragment when 'Y'
C1	Category selected	category = ?
C2	Brand(s) selected	brand IN (?, ...)
C3	Price range selected	price BETWEEN ? AND ?
C4	In-stock only	in_stock = 1
C5	Exclude colour	NOT (color = ?)

3. Decision table: all $2^5 = 32$ combinations

Rule	C1	C2	C3	C4	C5	Expected WHERE clause
1	N	N	N	N	N	(no WHERE filter: all products)
2	N	N	N	N	Y	NOT color = ?
3	N	N	N	Y	N	in_stock = 1
4	N	N	N	Y	Y	in_stock = 1 AND NOT color = ?
5	N	N	Y	N	N	price BETWEEN
6	N	N	Y	N	Y	price BETWEEN AND NOT color = ?
7	N	N	Y	Y	N	price BETWEEN AND in_stock = 1
8	N	N	Y	Y	Y	price BETWEEN AND in_stock = 1 AND NOT color = ?
9	N	Y	N	N	N	brand IN (...)
10	N	Y	N	N	Y	brand IN (...) AND NOT color = ?
11	N	Y	N	Y	N	brand IN (...) AND in_stock = 1
12	N	Y	N	Y	Y	brand IN (...) AND in_stock = 1 AND NOT color = ?
13	N	Y	Y	N	N	brand IN (...) AND price BETWEEN
14	N	Y	Y	N	Y	brand IN (...) AND price BETWEEN AND NOT color = ?
15	N	Y	Y	Y	N	brand IN (...) AND price BETWEEN AND in_stock = 1
16	N	Y	Y	Y	Y	brand IN (...) AND price BETWEEN AND in_stock = 1 AND NOT color = ?
17	Y	N	N	N	N	category = ?

Rule	C1	C2	C3	C4	C5	Expected WHERE clause
18	Y	N	N	N	Y	category = ? AND NOT color = ?
19	Y	N	N	Y	N	category = ? AND in_stock = 1
20	Y	N	N	Y	Y	category = ? AND in_stock = 1 AND NOT color = ?
21	Y	N	Y	N	N	category = ? AND price BETWEEN
22	Y	N	Y	N	Y	category = ? AND price BETWEEN AND NOT color = ?
23	Y	N	Y	Y	N	category = ? AND price BETWEEN AND in_stock = 1
24	Y	N	Y	Y	Y	category = ? AND price BETWEEN AND in_stock = 1 AND NOT color = ?
25	Y	Y	N	N	N	category = ? AND brand IN (...)
26	Y	Y	N	N	Y	category = ? AND brand IN (...) AND NOT color = ?
27	Y	Y	N	Y	N	category = ? AND brand IN (...) AND in_stock = 1
28	Y	Y	N	Y	Y	category = ? AND brand IN (...) AND in_stock = 1 AND NOT color = ?
29	Y	Y	Y	N	N	category = ? AND brand IN (...) AND price BETWEEN
30	Y	Y	Y	N	Y	category = ? AND brand IN (...) AND price BETWEEN AND NOT color = ?
31	Y	Y	Y	Y	N	category = ? AND brand IN (...) AND price BETWEEN AND in_stock = 1
32	Y	Y	Y	Y	Y	category = ? AND brand IN (...) AND price BETWEEN AND in_stock = 1 AND NOT color = ?

Rules are numbered from all N (rule 1) to all Y (rule 32). Each rule is executed with a known data set, and the result set is compared with an independently computed expected set.

4. Additional tests for logic inside the filters

- Brand filter: 1 brand, 2 brands, many brands (OR within group); verify that adding a brand never reduces results.
- Mixed operators: brand OR group plus price AND (precedence check; this exposes the Q5 style defect).
- Price boundaries: min = max, min > max (error or empty), 0, price exactly at min and max (inclusive).
- NOT: exclude a colour that no product has; exclude the only colour of the category (empty result).
- Empty result, sorting and pagination with filters, special characters in values (parameterized, no injection).

5. Evaluation of completeness

Aspect	Coverage	Remaining limitation
Filter combinations	100% of the 32 inferred combinations	Depends on the 5 filters being the complete list
Operator logic (AND/OR/NOT)	Covered by rules plus extra tests	Inferred from sample queries only
Boundaries	Price range and brand count covered	Other numeric filters (rating) need the same treatment
Model validity	Can be validated by replaying logged queries and comparing predicted WHERE clause	Unlogged behavior stays unverified

Conclusion: the model is complete with respect to the observed filter set. The 32-rule table is cheap to automate as parameterized tests, and a smoke subset of 8 rules (for example all N, all Y and each single filter) can be run on every build.

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation